RICE UNIVERSITY

T3 · Scale out: train on many workers

Retained prototype: combine gradients across MPI ranks and two GPUs; shard one AdamW update.

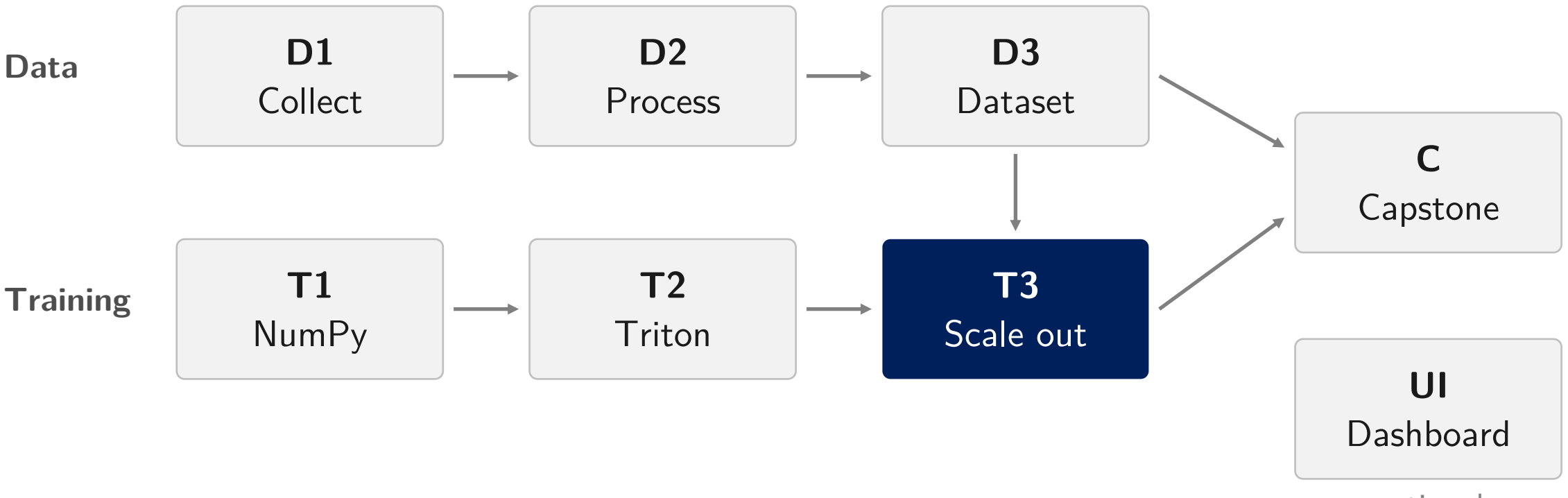
COMP 329 course staff

Rice University

Spring 2027 · Assignment T3

Release week 12 · Checkpoint week 13 · Final week 14 · 10-12 hours

Where this assignment fits



In our pipeline

This assignment: `runs/inputs/bpe` → **T3** → `runs/t3-cpu`, `runs/t3-gpu`

What you will build, and why

Several workers split one global batch but must take exactly the update a single worker would.

1. **Correctness first**

   Uneven batches make naive averaging wrong.

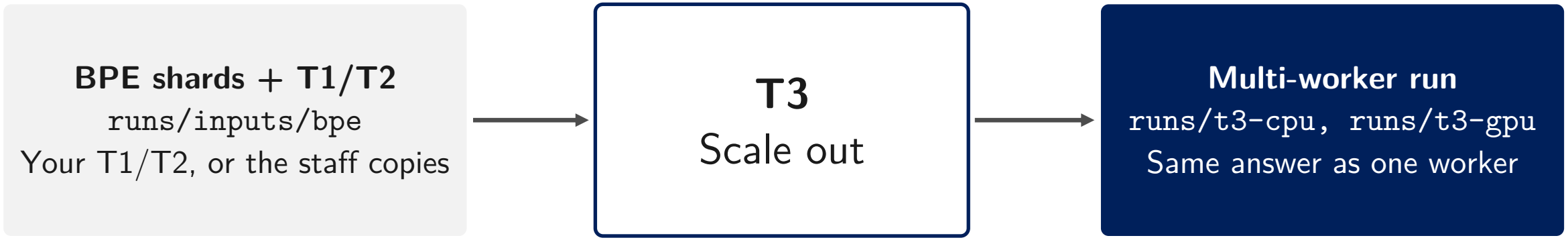
2. **Real collectives**

   Actual MPI on CPUs and NCCL on two GPUs.

3. **Memory sharding**

   Each rank keeps only its slice of the optimizer state.

What goes in, what comes out



Files you produce

`metrics.json`
`world_size`, collective, correctness, `sharded_adamw`

`checkpoint.npz`
Written by rank 0 only

`Collective benchmark`
time $= \alpha + \beta \cdot$ `payload_bytes`, fitted locally

Three collective operations

All-reduce SUM

`[a] + [b] → [a+b]` on every rank

Used to combine gradients.

Reduce-scatter

`Sum, then each rank gets one segment`

Start of the sharded update.

All-gather

`Join segments; everyone gets all`

End of the sharded update.

Key idea

Every rank must call the same collectives in the same order, with matching shapes and types, or the job can hang.

Why averaging rank means can be wrong

```
global batch B = 17, two ranks
rank 0: 9 examples, mean loss 2
rank 1: 8 examples, mean loss 4

wrong:  (2 + 4) / 2        = 3
right:  (9·2 + 8·4) / 17   = 50/17

weight each local mean by n_r / B,
then SUM all-reduce
```

1. **`Split with np.array_split`**

   Every rank sees the same global batch; reject an empty slice.

2. **Weight, then sum**

   The same weighting applies to the gradients.

3. **Pack in parameter order**

   Flatten, one collective, then unpack the shapes.

4. **Synchronize the GPU**

   NCCL calls are asynchronous; the loop clips after you return.

One sharded AdamW update

319

real parameters

160

shard length per rank

1280 B

moment state per rank (vs. 2552)

Key idea

Pad to equal shards, reduce-scatter, update your slice with both moments, all-gather, then drop the padding before comparing with a serial update. Error must be below 5e-5.

What you write, what you get

You write

`Reducer.__call__`

`starter/distributed.py` · 4 numbered TODOs

`verify_sharded_adamw`

`starter/distributed.py`

Provided

- Process and device bootstrap
- Serial update checks
- Collective benchmark, MPI entry point
- T1 and T2: yours, or staff copies in provided/

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1 — 2 — 3 — 4 — 5

Collectives

All-reduce, reduce-scatter, all-gather.

Reducer

Weighted local means; one SUM collective.

Sharded AdamW

One update that matches the serial one.

Final runs

Two/three CPU ranks and two GPUs.

Scaling study

Fixed global work; include communication.

```
check t3 --checkpoint --systems
```

```
check t3 --systems
```

```
lab.py t3
```

Check

Done when: All 4 tests pass with real MPI and NCCL, including the batch of 17.

If something goes wrong

Even batches work; 17 fails
Check: Weight by example count, not rank count.

Only some shapes differ
Check: Same packing order and offsets on every rank.

MPI job hangs
Check: Branches, collective order, buffer shapes, error logs.

Two ranks use one GPU
Check: Keep the supplied local-rank device mapping.

Sharded error has an extra value
Check: Remove padding before comparing.

Checkpoint writers conflict
Check: Only rank 0 writes files.

The performance study

Do more ranks make training faster?

Predict first

Gradient payload = 4 bytes × parameters, plus collective overhead. Predict where communication dominates.

CPU ranks 1 / 2 / 4
global batch 64, 40 steps

GPU ranks 1 / 2
same model, data, seed, order

How to measure: One warmup + three repeats. Report targets/s, training, communication and setup time, speedup and efficiency.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: weighted reducer 12 · distributed training 10 · sharded update 8 · individual walkthrough 10

Check

Done when: All 4 tests pass with real MPI and NCCL, including the batch of 17.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py inputs --output runs/inputs
$ python course.py check t3 --checkpoint \
    --systems
$ python lab.py t3 --input runs/inputs/bpe \
    --output runs/t3-cpu --workers 2 \
    --batch-size 17 --backend numpy
```

Your folder

`06_t3_scale_out/`

Handout:
`06_t3_scale_out/README.md`